

Unified Resource Booking System

Database Systems Course Project · Group 25

Sahaj Jhamb (Group Leader) · Arihant Bansal · Tejas Bhat · Amrit Kumar Ghoshal · Pranav Prafulla Aherrao · Nikshay Jain

Abstract

This project presents a functional, full-stack web platform designed to streamline and bring transparency to the booking of shared campus resources — classrooms, laboratories, equipment, and seminar halls — for classes, evaluations, meetings, and departmental activities. The system manages submission, routing, and multi-faculty approval of booking requests through a structured role-based workflow, eliminating scheduling conflicts and manual coordination overhead while providing real-time status visibility to all stakeholders.

1. SYSTEM OVERVIEW — TECHNOLOGY STACK

Layer	Technology	Description
Frontend	HTML5 · CSS3 · Bootstrap 5 · JavaScript	Seven Jinja2 templates (login, signup, dashboard, booking form, faculty dashboard, admin panel, summary) with Bootstrap 5 responsive layout. Client-side JS handles form validation, password-match indicator, date-floor, and dynamic approval hints.
Backend	Python 3 · Flask · SQLAlchemy ORM · Gunicorn · WhiteNoise	22 Flask routes protected by three role decorators. Gunicorn is the production WSGI server on Render. WhiteNoise middleware is mounted directly on the WSGI app to serve /static/ files from Gunicorn without a separate web server or CDN.
Database	PostgreSQL — Render Managed Cloud	Eleven-table schema with FK constraints and ENUM types. Developed and tested locally using a raw SQL schema, then migrated to a cloud-hosted PostgreSQL instance on Render for production. SQLAlchemy ORM keeps the codebase database-agnostic.
Deployment	Render · render.yaml · Environment Variables	Declared via render.yaml. DATABASE_URL, SECRET_KEY, and ANTHROPIC_API_KEY are injected as Render environment variables — nothing hardcoded. db.create_all() initialises tables at startup; seed.py populates demo data once.
AI Feature	Anthropic Claude API — auxiliary, read-only	Booking statistics are queried from PostgreSQL and sent to claude-sonnet-4-6 for a natural-language summary. Strictly read-only. Falls back to a deterministic rule-based summary when the API key is absent.

2. USER ROLES & PRIVILEGES

Role	Credentials	Capabilities	Restrictions
Student	alice@college.edu pass123	Register, log in, browse resources, submit bookings, view own booking history with per-faculty approval progress and rejection reasons.	Cannot view others' bookings, access admin panel, faculty dashboard, or summary page. No cancellation.
Faculty	bob@college.edu pass123	All student rights plus Faculty Approval Dashboard: view pending requests for assigned resources, approve/reject with reason, view personal action history.	Cannot manage resources, configure rules, assign faculty, or act outside assigned resources.
augsd (Super Admin)	augsd@college.edu augsd123	Full access: add/toggle resources, set ANY/ALL rules per resource, manage faculty assignments, override any booking, view all bookings system-wide, access Summary and AI Insights.	No restrictions. Only role with Booking Summary tab visibility.

3. TEAM MEMBER CONTRIBUTIONS

Sahaj Jhamb

GROUP LEADER

ER Diagram & Normalisation · Project Lead

ER Design · 3NF Normalisation · System Architecture · Integration

Designed the complete ER diagram covering all 11 entities: departments, roles, users, resource types, resources, time slots, bookings, approval rules, faculty mappings, approvals, and usage logs. Applied 3NF normalisation — separating ResourceApprovalRule and ResourceFacultyMapping into independent tables directly enabled the ANY/ALL approval engine without redundancy. As group leader, coordinated all module handoffs, enforced naming consistency across SQL schema, ORM models, routes and templates, and managed end-to-end integration.

Arihant Bansal

Triggers & Stored Procedures

BEFORE INSERT Trigger · Stored Procedures · SQL View · Conflict Detection

Implemented database-level automation: (1) prevent_double_booking — a BEFORE INSERT trigger raising SQLSTATE '45000' when the same resource, slot, and date already holds a pending or approved booking. (2) GetBookingsByResource(p_resource_id) — a stored procedure joining bookings with users, time_slots, and resources for resource-specific lookups, demonstrated live on the Summary page. (3) booking_schedule — a six-table SQL view used across the dashboard and summary.

Tejas Bhat

Backend Engineering & Cloud Deployment

Flask · SQLAlchemy · Render · PostgreSQL · Gunicorn · WhiteNoise

Built all 22 Flask routes: authentication, booking creation with conflict detection, faculty approval/rejection, augsd override, resource management, and AI Insights. Authored the core _process_approval_outcome() engine evaluating ANY/ALL logic on every faculty action and auto-generating UsageLog entries. Led Render deployment: render.yaml, Gunicorn WSGI config, WhiteNoise static serving, postgres:// URI fix, and full environment-variable-driven credential management.

Amrit Kumar Ghoshal

Analytic Queries & SQL Views

SQL Views · Aggregate Queries · GROUP BY · Dashboard Analytics

Designed the booking_schedule SQL view — a six-table join across bookings, users, departments, resources, resource_types, and time_slots — providing a unified reporting surface for the dashboard and summary pages. Wrote all analytic queries: total booking count, status breakdown via GROUP BY, top-N resources by frequency, and usage distribution data consumed by both the Summary page and the AI Insights endpoint as statistical context for the Claude API prompt.

Pranav Prafulla Aherrao

SQL Schema & Constraints

DDL Schema · Foreign Keys · ENUM Types · Constraints · DBA Privileges

Translated the ER diagram into a production-ready SQL schema: 11 tables with primary keys, cascaded foreign-key relationships, NOT NULL and UNIQUE constraints on emails and department names, ENUM types for booking status (pending/approved/rejected/cancelled) and rule type (ANY/ALL), and DEFAULT values for timestamps and flags. Authored the DBA privilege script demonstrating three tiers: read-only, read-update (SELECT + UPDATE on status, no DDL), and full admin — with SHOW GRANTS verification.

Nikshay Jain

Frontend Development & Documentation

HTML5 · Bootstrap 5 · Jinja2 · JavaScript · Documentation

Built all seven Jinja2 templates: login with credential hints, signup with role and department dropdowns, student dashboard showing per-faculty approval progress and rejection reasons, booking form with dynamic approval hint on resource selection, faculty dashboard with pending actions and history, augsd admin panel with tabbed resource/rules/bookings management, and the augsd-only summary page. Added JS validation across all forms and authored the full project documentation.

4. MODULES DEVELOPED

ID	Module	Description
M1	Authentication	Session-based login/signup with email uniqueness check, 6-char password minimum, and role-restricted self-registration (augsd via seed.py only). Three route decorators (login_required, faculty_required, augsd_required) enforce access control on all routes.
M2	Resource Booking	Core booking flow: resource/slot/date selection with server-side conflict detection. Creates one Approval row per mapped faculty for approval-required resources (status=pending), or immediately approves and logs usage otherwise. Form dynamically shows rule type + faculty count.
M3	Approval Engine	_process_approval_outcome() re-evaluates status after every faculty action using the resource's ANY/ALL rule. A single rejection immediately sets the booking to rejected and stores the reason on the Booking record for display to the requesting user.
M4	Faculty Dashboard	Faculty-exclusive view of pending approvals for assigned resources only — full booking context, approve/reject form with reason field, personal action history (last 20), and assigned resource list with rule-type badge. Strict resource-level isolation.
M5	augsd Admin Panel	Three-tab superuser interface: Resources (add/activate/deactivate), Rules (set ANY/ALL per resource, assign/remove faculty), Bookings (full system list with one-click override that stamps all pending Approval rows for audit consistency).
M6	Summary & Analytics	augsd-only page. Metric cards for total/approved/pending/rejected counts, top-5 resource ranking by frequency, and recent-50 booking table — all driven by SQLAlchemy GROUP BY aggregate queries (equivalent to the booking_schedule SQL view).
M7	AI Insights	Queries three stats from PostgreSQL and sends them to claude-sonnet-4-6 (Anthropic API) for a natural-language summary. HTTP errors surfaced as readable messages. Falls back to a deterministic rule-based summary when the API key is absent. Strictly read-only.
M8	Database & Integrity	11-table PostgreSQL schema via SQLAlchemy ORM with FK chains, ENUM constraints, and auto-generated UsageLog on approval. db.create_all() runs at startup; seed.py populates demo data idempotently. URI prefix fix ensures Render's postgres:// works with SQLAlchemy.

5. PROJECT LINKS & REPOSITORY

Source Code — GitHub Repository

[github.com/Sahaj47/
unified-booking-system](https://github.com/Sahaj47/unified-booking-system)

Complete source code, SQL schema, seed data, render.yaml, and all Flask/HTML/CSS files. Clone and configure a .env file with PostgreSQL credentials, then run seed.py once to get started locally.

Live Deployment — Render

[unified-booking-system-hjhfh.
onrender.com/login](https://unified-booking-system-hjhfh.onrender.com/login)

Publicly accessible live deployment. Use the demo credentials on Page 1 to sign in as student, faculty, or augsd superuser.

Note — Render Free Tier Cold Start Behaviour

Render's free-tier web services spin down after approximately 15 minutes of inactivity. The first visit after an idle period may take up to 30–60 seconds to load — please wait rather than refreshing immediately. If a 500 Internal Server Error appears on the very first load (caused by the process being overwhelmed at startup), a single reload after a few seconds resolves it. All subsequent requests in the session will respond promptly.